Compito 7.1 APA

Fabio Nicolosi s7592267@studenti.unige.it

May 2026

1 Introduzione - Spiegazione del algoritmo

```python
# true --> composto
# false --> non composto
def MCprimalityTest_aux(a:int,s:int,q:int | float,n:int):
    x = pow(a,int(q),n) # da solo valori tra (0) - (n-1)

    # 1 --> se il risultato del modulo è 1
    # n-1 --> se il risultato del modulo è -1 dato che n-1 è il rappresentate dei elementi che danno -1
    # come risultato del modulo
    if(x == 1 or x== n-1):
        print(f"{a}: Non composto")
        return True

    while s-1>=0:
        x= pow(x,2,n)
        if(x== n-1):
            print(f"{a}: Non composto")
            return True
        else:
            s = s-1
    print(f"{a}: composto")
    return False


def MCprimalityTest(n:int):
    if(n < 3 or n%2 == 0):
        print(f"{n}: non è dispari ne maggiroe di 3")
        return

    s = 0
    q = n-1
    while(q%2 == 0):
        s=s+1
        q=q/2

    print(f"Dato {n}:\n\ts:{s}\n\tq={q}\n")
    print(f"--------- OUTPUT n:{n} ---------")
    for a in range(2, n-1):
        ris[a] = MCprimalityTest_aux(a,s,q,n)
    print(f"--------- FINE ---------")
```

Implementazione

Come mostrato nell'immagine soprastante, l'algoritmo `MCPrimalityTest(n)` è stato implementato in Python e suddiviso in due componenti principali:

1.1 Riscrittura del numero dispari e validazione dell'input

Nella prima parte della funzione `MCPrimalityTest(n)`, vengono eseguite due operazioni fondamentali. In primo luogo, si verifica che il numero n ricevuto in input sia dispari e maggiore o uguale a 3; in caso contrario, l'input viene scartato. Questa validazione è necessaria poiché la teoria garantisce che ogni numero dispari $n \geq 3$ può sempre essere espresso nella forma:

$$n = q \cdot 2^s \quad \text{con } s \geq 1 \text{ e } q \text{ dispari}$$

Il ciclo `while` iniziale ha lo scopo di calcolare i parametri q e s. Il procedimento prevede di inizializzare $q = n - 1$ e $s = 0$. Successivamente, finché q rimane un numero pari, esso viene diviso per 2 e il valore di s viene incrementato di una unità. Una volta terminato il ciclo, i parametri calcolati vengono passati alla funzione ausiliaria *MCPrimalityTest_aux(n)* insieme alla base a.

1.2 Test di primalità per una base a

La funzione `MCPrimalityTest_aux(a, s, q, n)` costituisce il nucleo del test probabilistico per determinare se il numero sia composto o meno. Per ciascuna base $a \in \{2, 3, \ldots, n-2\}$, l'algoritmo calcola inizialmente il valore $x \equiv a^q \pmod{n}$. Se questo valore appartiene alla classe di congruenza di 1 o di -1 (rappresentata da $n-1$), l'algoritmo decreta che il numero è "*probabilmente primo*".
Se la condizione iniziale non è soddisfatta, si entra in un ciclo che effettua quadrature successive finché la condizione $s - 1 > 0$ resta valida:

- Si aggiorna il valore calcolando $x \equiv x^2 \pmod{n}$.
- Si verifica se x è congruente a $-1 \pmod{n}$. Se la condizione è vera, l'algoritmo si interrompe e restituisce che n è "probabilmente primo".
- Se la condizione è falsa, si decrementa s di una unità e si procede con la quadratura successiva.

Qualora il ciclo si esaurisca senza che nessuna delle condizioni precedenti sia stata soddisfatta, si deduce con certezza matematica che n è un *numero composto*.

2 Fondamenti teorici

L'algoritmo di Miller-Rabin si fonda su due teoremi cardine della teoria dei numeri:

2.1 Teorema di Eulero

Se $a \in \mathbb{Z}_n^*$, dove n è un intero positivo, allora vale la relazione:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

In altri termini, ogni elemento appartenente al gruppo moltiplicativo modulo n (cioè ogni intero coprimo con n) , se elevato alla funzione $\phi(n)$ (la quale indica la cardinalità degli elementi invertibili in $\mathbb{Z}_n$), risulta congruente a 1 modulo n.

2.2 Piccolo Teorema di Fermat

Se p è un numero primo e a è un intero non divisibile per p, allora si ha che:

$$a^{p-1} \equiv 1 \pmod{n}$$

Dato che per un numero primo p la funzione di Eulero assume il valore $\phi(p) = p - 1$, questo teorema non è altro che un caso speciale del Teorema di Eulero.

2.3 Sottogruppi

Dato un gruppo G, un suo sottoinsieme non vuoto H si definisce sottogruppo di G (indicato con $H \subseteq G$) se soddisfa le seguenti proprietà strutturali:

- **Chiusura**: Per ogni coppia di elementi $a, b \in H$, il loro prodotto deve appartenere al sottogruppo, ovvero $a \cdot b \in H$.
- **Elemento neutro**: L'elemento neutro del gruppo G (che nel caso del gruppo moltiplicativo $\mathbb{Z}_n^*$ è l'elemento 1) deve appartenere ad H.
- **Inversi**: Per ogni elemento $a \in H$, anche il suo inverso moltiplicativo deve appartenere ad H, ovvero $a^{-1} \in H$.

2.4 Teorema di Lagrange

Il Teorema di Lagrange stabilisce una proprietà fondamentale per i gruppi finiti : se H è un sottogruppo di un gruppo finito G, allora l'ordine (la cardinalità) di H deve dividere l'ordine di G. Matematicamente la relazione si esprime come:

$$|G| = n \cdot |H| \quad \text{con } n \in \mathbb{N}^+$$

Dove il coefficiente n rappresenta l'indice del sottogruppo H all'interno di G (spesso indicato con $[G : H]$), il quale esprime il numero di classi laterali distinte in cui il gruppo viene partizionato.
La prima e fondamentale implicazione di questo teorema è che la cardinalità del sottogruppo H è sempre un divisore della cardinalità del gruppo principale G. Inoltre, qualora l'ordine di H sia strettamente minore dell'ordine di G (ovvero $|H| < |G|$), si deduce che H è un sottoinsieme proprio di G (indicato con $H \subset G$); questo significa che ogni elemento appartenente ad H è contenuto anche in G, ma esistono elementi di G che non trovano corrispondenza all'interno del sottogruppo H.

```python
def check (sG: list[int], G:list[int]):

    for x in sG:
        for y in sG:
            mod = ((x*y) % 65)
            if( mod not in sG):

                return f"Non è un gruppo: {x} * {y} mod 65 = {mod} "

    if len(G) % len(sG) != 0:
        return "E un Gruppo, ma non è sottogruppo di Zn"
    else:
        return "E sottogruppo"
```

Veirifca del sottogruppo

2.5 Testimoni e Bugiardi

```
MCprimalityTest(65)


for a, res in ris.items():
    if res :
        bugiardi.append(a)

for a in range(2,64):
    if math.gcd(a, 65) > 1:
        test_B.append(a)

for a, res in ris.items():
    if math.gcd(a, 65) == 1 and not res:
        test_NotB.append(a)
```

Find bugiardi

Per comprendere l'affidabilità e il funzionamento dell'algoritmo di Miller-Rabin, è necessario introdurre due definizioni fondamentali: i *MR-Testimoni* e i *MR-Bugiardi*.

- **MR-Testimoni**: Si definiscono testimoni tutte le basi a che, elaborate dall'algoritmo, producono come output "composto". Se una base si comporta come testimone, essa fornisce la certezza matematica che il numero n sotto esame non è primo.
- **MR-Bugiardi**: Si definiscono bugiardi tutte quelle basi a che, nonostante il numero n sia realmente composto (come si vedrà in seguito nell'analisi dell'output per $n = 65$), inducono l'algoritmo a produrre l'output "probabilmente primo". In questo caso, la base "inganna" il test fornendo un falso positivo.

I **MR-Testimoni** (coloro che dicono la verità, dimostrando che il numero è composto) possono essere a loro volta suddivisi in due categorie:

2.5.1 Testimoni Banali

I testimoni banali sono tutte quelle basi a per le quali il Massimo Comun Divisore con il numero da testare è maggiore di 1, ovvero $\text{MCD}(a, n) > 1$. In questo caso, non è nemmeno necessario avviare il calcolo delle potenze dell'algoritmo di Miller-Rabin per scoprire che il numero è composto: la base a condivide un fattore con n e, di conseguenza, a non appartiene al gruppo moltiplicativo $\mathbb{Z}_n^*$

2.5.2 Testimoni Non Banali

I testimoni non banali sono invece basi che risultano coprime con n (ovvero $\text{MCD}(a, n) = 1$), ma che riescono comunque a dimostrare la non-primalità del numero attraverso il fallimento delle quadrature successive. Una base è un testimone non banale se la sequenza dei resti generata non rispetta le proprietà tipiche dei numeri primi.

Esempio di codice bugiardo non banale:
Dato $n = 65$, da cui si ricavano: $s = 6$ e $q = 1$
Per la base $a = 8$ abbiamo che:

$$8^1 \equiv 8 \pmod{65}$$
$$8^2 = 64 \equiv -1 \pmod{65}$$

Poiché alla seconda quadratura otteniamo -1, l'algoritmo si ferma e dichiara erroneamente che 65 è "*non composto*". La base 8 è quindi un MR-bugiardo non banale.

3 Analizi dei dati

```
Dato 65:
        s:6
        q=1

--------- OUTPUT n:65 ---------
2: composto
.
.
.
7: composto
8: Non composto
9: composto
.
.
.
18: Non composto
19: composto
20: composto
21: composto
.
.
.

46: composto
47: Non composto
48: composto
.
.
.
56: composto
57: Non composto
58: composto
.
.
.
63: composto
--------- FINE ---------

Bugiardi (nell'intervallo 2-63):  [8, 18, 47, 57]

Testimoni Banali:  [5, 10, 13, 15, 20, 25, 26, 30,
                    35, 39, 40, 45, 50, 52, 55, 60]

Testimoni NON Banali:  [2, 3, 4, 6, 7, 9, 11, 12, 14, 16, 17,
                        19, 21, 22, 23, 24, 27, 28, 29, 31, 32,
                        33, 34, 36, 37, 38, 41, 42, 43, 44, 46, 48,
                        49, 51, 53, 54, 56, 58, 59, 61, 62, 63]

Z*_n:  [1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 14, 16, 17, 18, 19,
        21, 22, 23, 24, 27, 28, 29, 31, 32, 33, 34, 36, 37,
        38, 41, 42, 43, 44, 46, 47, 48, 49, 51, 53, 54, 56,
        57, 58, 59, 61, 62, 63, 64]

Insieme H_65 con definzione:  [1, 8, 12, 14, 18, 21, 27, 31, 34,
                               38, 44, 47, 51, 53, 57, 64]

E sottogruppo  Verificato su:
        Gruppo in eseme: [1, 8, 12, 14, 18, 21, 27, 31, 34,
                          38, 44, 47, 51, 53, 57, 64]
        Gruppo: Z*_n (vedi sopra)
```

A partire dagli output generati dall'esecuzione del codice per il numero $n = 65$ — il quale è notoriamente un numero composto (5×13) — è possibile tracciare un'analisi dettagliata del comportamento delle basi verificate:

3.1 Analisi dei MR-Bugiardi

Come si evince dai risultati empirici, l'algoritmo individua un insieme specifico di basi composto dagli elementi $\{8, 18, 47, 57\}$. Nonostante il numero 65 sia composto, l'elaborazione di queste quattro basi induce il test a restituire l'esito "non composto" (ovvero probabilmente primo), configurandosi a tutti gli effetti come falsi positivi o MR-bugiardi non banali

3.2 Analisi dei MR-Testimoni

L'output del programma suddivide correttamente le restanti basi dell'intervallo $\{2, 3, \ldots, 63\}$ in due sottoinsiemi ben distinti:

- Testimoni Banali: Raccolgono tutti quei valori che condividono un fattore comune con 65 (come il 5 o il 13 e i loro multipli), presentando un $\text{MCD}(a, 65) > 1$.
- Testimoni Non Banali: Rappresentano la maggioranza delle basi analizzate; sono elementi coprimi con 65 ($\text{MCD}(a, 65) = 1$) ma i cui resti, sottoposti a quadrature successive, violano la sequenza attesa per i numeri primi, dimostrando matematicamente la componibilità del numero.

3.3 Il Gruppo Moltiplicativo $\mathbb{Z}_{65}^*$

Attravento l'automazione del software è stato generato l'insieme degli elementi che compongono il gruppo moltiplicativo $\mathbb{Z}_{65}^*$, definito come:

$$\mathbb{Z}_{65}^* = \{x \in \mathbb{Z}_{65}^+ \setminus \{0\} \mid \text{MCD}(x, 65) = 1\}$$

Questo insieme comprende tutti e soli i numeri coprimi con n e ammette una cardinalità complessiva di $\phi(65) = 48$ elementi. È di fondamentale importanza teorica sottolineare che, qualora il valore n inserito in input fosse stato un numero primo, il gruppo moltiplicativo avrebbe incluso la totalità degli elementi compresi nell'intervallo tra 1 e $n - 1$.

3.4 Il Sottogruppo H_n e la Verifica algebrica

Sulla base della definizione matematica dell'insieme H_n, inteso come il sottoinsieme degli elementi di $\mathbb{Z}_n^*$ che soddisfano il criterio del Piccolo Teorema di Fermat:

$$H = \{x \in \mathbb{Z}_{65}^* \mid x^{64} \equiv 1 \pmod{65} \}$$

Il programma ha popolato e ricavato i rispettivi componenti. Tramite uno script di controllo, sono state testate e verificate tutte le proprietà algebriche fondamentali precedentemente descritte (chiusura rispetto al prodotto modulare, esistenza dell'elemento neutro e presenza degli inversi moltiplicativi). L'esito del test permette di affermare che H_{65} costituisce a tutti gli effetti un sottogruppo proprio di $\mathbb{Z}_{65}^*$. Trovando perfetta corrispondenza con il Teorema di Lagrange, si osserva che la cardinalità del sottogruppo ($|H_{65}| = 16$) divide esattamente la cardinalità del gruppo principale ($|\mathbb{Z}_{65}^*| = 48$), con un indice pari a $48/16 = 3$

4 Conclusioni

Algoritmo 7.1. *MCPrimalityTest*(n)
Input: $n \geq 3$ dispari
Output: n `composto` o `probabilmente primo`

---

1. $s = 0$
2. $q = n - 1$
3. `while` (q pari)
   $s \leftarrow s + 1$
   $q \leftarrow q/2$
4. campiona a uniformemente a caso in $\{2, 3, \ldots, n-2\}$
5. calcola $x \equiv a^q \pmod{n}$
6. `if` $x \equiv \pm 1$
   `return` n `probabilmente primo`
7. `while` $s - 1 \geq 0$
   $x \equiv x^2 \pmod{n}$
   `if` $x \equiv -1 \pmod{n}$
   `return` n `probabilmente primo`
   $s \leftarrow s - 1$
8. `return` n `composto`

Pseudocodice

Prima di trarre le conclusioni, è opportuno precisare una nota metodologica: gli output analizzati nella sezione precedente derivano da un'implementazione software che non seleziona le basi a in modo casuale — come previsto dal protocollo probabilistico originale — bensì esegue un ciclo esaustivo su tutti i valori possibili. Questa scelta progettuale è stata adottata al fine di mappare e popolare l'intera struttura degli insiemi di interesse con la massima accuratezza possibile.
Come ampiamente dimostrato, l'esistenza dei MR-bugiardi implica che l'esecuzione del test su una singola base casuale nell'intervallo $\{2, 3, \ldots, n-2\}$ non consenta di determinare a priori se l'esito "non composto" sia un falso positivo. Tuttavia, l'efficacia del test di Miller-Rabin poggia su due pilastri algebrici e probabilistici:

4.1 La struttura algebrica dei sottogruppi

L'insieme H_n, definito come il sottoinsieme degli elementi di $\mathbb{Z}_n^*$ che soddisfano il criterio di Fermat, contiene la totalità delle basi bugiarde. Essendo H_n un sottogruppo proprio di $\mathbb{Z}_n^*$, il Teorema di Lagrange impone che la cardinalità di H_n divida esattamente la cardinalità del gruppo principale $G = \mathbb{Z}_n^*$ per un coefficiente intero x (detto indice del sottogruppo):

$$|H_n| = \frac{\mathbb{Z}_n^*}{x}$$

Poiché H_n è un sottogruppo proprio ($H_n \subset \mathbb{Z}_n^*$), l'indice x deve essere un numero intero maggiore o uguale a 2 ($x \geq 2$). Da questa proprietà discende che, persino nel peggiore dei casi teorici, la dimensione del sottogruppo non può superare la metà del gruppo complessivo:

$$|H_n| = \frac{\mathbb{Z}_n^*}{2}$$

4.2 Amplificazione della probabilità

Questa restrizione algebrica garantisce che almeno la metà degli elementi di $\mathbb{Z}_n^*$ si configuri come un MR-testimone non banale. Di conseguenza, effettuando un singolo campionamento uniformemente a caso, la probabilità di selezionare un testimone in grado di dimostrare la componibilità del numero è pari ad almeno $\frac{1}{2}$ Per azzerare l'incertezza dovuta ai bugiardi, il test viene ripetuto in modo indipendente per un numero k di volte. La probabilità che un numero composto superi erroneamente tutti i controlli, venendo dichiarato "probabilmente primo" per k volte consecutive, decresce esponenzialmente secondo la relazione:

$$\mathcal{P}(\text{Errore}) \leq \left(\frac{1}{2}\right)^k$$

Scegliendo un valore di k adeguatamente elevato (ad esempio $k = 40$), la probabilità di errore diventa talmente infinitesimale da conferire all'algoritmo Monte Carlo una sicurezza e un'affidabilità